Cheat Sheet: Agent Eval CI Gate (Promptfoo + LangGraph)

Source(s): W05/D2/eval-ci-lab (provider.py, trajectory.py, fake_supervisor.py, promptfooconfig.yaml, eval-gate.yml)

Date: 2026-06-24

1. Big Picture

- A **CI gate** that blocks a PR when an agent regresses on its *behavior*, not just its answer.
- It checks the agent's **trajectory** (which workers ran, in what order), **termination** (did it finish / no loops), **token budget**, and **output content**.
- Built on **Promptfoo** — a config-driven eval runner ("pytest for LLM outputs"). One YAML file = the whole gate.
- The load-bearing trick: a Python **provider** runs the agent and returns the answer **plus** structured trajectory metadata, so assertions can judge the *path*.
- Runs a **deterministic fake** agent in CI (free, no API keys) and the **real** agent on demand (opt-in via env).

2. Mental Model

It's a factory QC line bolted onto your merge button. Promptfoo is the inspector, the provider is the conveyor that brings each agent run to the inspector with a full work-history tag attached, and the **exit code** is the stamp: 0 = pass (merge), non-zero = reject (block).

3. Key Concepts

Concept	Simple meaning	Remember it as
Promptfoo	OSS eval runner; runs cases, checks assertions, exits non-zero on fail	"pytest for LLM apps"
Provider	Python bridge: runs the agent, returns `output` + `metadata`	The adapter
Trajectory	Ordered list of workers that ran (from message `name` tags)	The agent's path
Assertion	A pass/fail rule (order, terminated, budget, contains)	The QC check
`context.metadata`	Where JS assertions read trajectory data (NOT `output`)	The gotcha
Exit code	0 = green/merge, non-zero (100) = red/block	The only signal CI reads
`FAKE_BROKEN`	Env switch to simulate a broken agent for testing	The fire drill

4. Key Patterns

- **Gate first, judge later** — deterministic checks now; LLM-as-judge + calibration come in D3.
- **Ordered-subsequence trajectory check**, not strict equality — a legit extra search loop must not cause a false red.
- **Fake-by-default in CI** — proves the gate's own logic for free; real agent is opt-in.
- **Zero changes to the agent repo** — provider imports the supervisor by path; tokens captured via a LangChain callback.

5. Diagram / Schema

```
GitHub Action (eval-gate.yml)   # runs on every PR
        ■
        ▼
promptfooconfig.yaml            # 8 cases + 4 assertion rules
        ■ calls file://provider.py
        ▼
provider.py ■■■ returns output (str) + context.metadata {worker_sequence, step_count, terminated, total_toker
   ■  ■■■ trajectory.py        # reads path from message name-tags
   ■  ■■■ fake_supervisor.py   # CI default (free) + FAKE_BROKEN switch
   ■        (or RUN_REAL_SUPERVISOR=1)
   ▼
W04 langgraph-lab/supervisor   # real agent, untouched
        ■
        ▼
   any assertion fails → exit 100 → job red → (with branch protection) MERGE BLOCKED
```

6. Critical Info (what NOT to get wrong)

- **The exit code is everything.** 0 = green, non-zero = red. Check with `echo $?`. CI reads only this.
- **In JS assertions,** `output` **is a STRING; metadata lives at** `context.metadata`**.** Using `output.metadata` silently breaks every check. (Verified empirically — don't assume.)
- **A red ✗ does NOT block merge by itself.** You must enable a **branch protection rule** requiring the `eval-gate` check. Until then the gate is advisory.
- **The gate only runs in the repo that holds the workflow file.** A PR in `langgraph-lab` won't trigger anything until the gate is ported there.
- **Don't** `npm install` **locally** — `npx promptfoo@latest` fetches it; `node_modules` is gitignored, never committed.
- **Trajectory = ordered subsequence**, so legitimate retry loops don't false-positive; only genuinely missing/extra steps fail.

7. Mini Example

```
cd eval-ci-lab
pip install -r requirements.txt

# healthy → 8 passed, exit 0 (green)
npx promptfoo@latest eval -c promptfooconfig.yaml ; echo "exit: $?"

# simulate a broken agent → fails, exit 100 (red)
FAKE_BROKEN=loop npx promptfoo@latest eval -c promptfooconfig.yaml ; echo "exit: $?"
```

```
#   → "step budget exceeded: 7 > 5"
```

Injected break	Fails	Caught by
skip_analyst	8/8	trajectory: `[search, writer]`
no_finish	8/8	termination: next != FINISH
loop	8/8	step budget: 7 > 5
empty_report	2/8	output missing `## Key Findings`

8. References

- `promptfooconfig.yaml` — the gate (cases + assertions)
- `eval_harness/provider.py` — agent↔promptfoo bridge (real | fake)
- `eval_harness/trajectory.py` — path reconstruction from message tags
- `eval_harness/fake_supervisor.py` — deterministic fixture + `FAKE_BROKEN`
- `.github/workflows/eval-gate.yml` — CI wiring (PR → eval → exit code)
- `proof_log.md` — recorded green/red evidence
- Promptfoo docs — assertions, providers, CI/CD